CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

Project Overview

time_based_todolist — A todolist application with time-based features.

This project is in early development (no source code yet). Update this file as the project grows.

bkit Development Context

- **Level**: Dynamic (fullstack with bkend.ai BaaS)
- **Pipeline Phase**: 1 (Schema definition)
- **PDCA Status**: `docs/.pdca-status.json`

Getting Started

Once the project is initialized, add the following sections here:

- Build/run commands
- Test commands
- Environment variable setup

CLAUDE.md

Behavioral guidelines to reduce common LLM coding mistakes. Merge with project-specific instructions as needed.

Tradeoff: These guidelines bias toward caution over speed. For trivial tasks, use judgment.

1. Think Before Coding

Don't assume. Don't hide confusion. Surface tradeoffs.

Before implementing:

- State your assumptions explicitly. If uncertain, ask.
- If multiple interpretations exist, present them - don't pick silently.
- If a simpler approach exists, say so. Push back when warranted.
- If something is unclear, stop. Name what's confusing. Ask.

2. Simplicity First

Minimum code that solves the problem. Nothing speculative.

- No features beyond what was asked.

- No abstractions for single-use code.
- No "flexibility" or "configurability" that wasn't requested.
- No error handling for impossible scenarios.
- If you write 200 lines and it could be 50, rewrite it.

Ask yourself: "Would a senior engineer say this is overcomplicated?" If yes, simplify.

3. Surgical Changes

Touch only what you must. Clean up only your own mess.

When editing existing code:

- Don't "improve" adjacent code, comments, or formatting.
- Don't refactor things that aren't broken.
- Match existing style, even if you'd do it differently.
- If you notice unrelated dead code, mention it - don't delete it.

When your changes create orphans:

- Remove imports/variables/functions that YOUR changes made unused.
- Don't remove pre-existing dead code unless asked.

The test: Every changed line should trace directly to the user's request.

4. Goal-Driven Execution

Define success criteria. Loop until verified.

Transform tasks into verifiable goals:

- "Add validation" → "Write tests for invalid inputs, then make them pass"
- "Fix the bug" → "Write a test that reproduces it, then make it pass"
- "Refactor X" → "Ensure tests pass before and after"

For multi-step tasks, state a brief plan:

```
1. [Step] → verify: [check]
2. [Step] → verify: [check]
3. [Step] → verify: [check]
```

Strong success criteria let you loop independently. Weak criteria ("make it work") require constant clarification.

These guidelines are working if: fewer unnecessary changes in diffs, fewer rewrites due to overcomplication, and clarifying questions come before implementation rather than after mistakes.

도구 호출 최소화 원칙

기본 규칙

- **파일 경로를 알고 있으면** `Read` 도구를 바로 사용한다. `find`나 `grep`으로 먼저 탐색하지 않는다.
- **심볼·문자열 위치를 알고 있으면** `Bash(grep)` 한 번으로 끝낸다. 여러 번 좁혀가며 검색하지 않는다.
- **편집 범위가 명확하면** `Read` → `Edit` 순서로 두 번에 완료한다. 확인용 `Read`를 반복하지 않는다.
- **독립적인 작업은 병렬 호출**로 처리한다. 순차적으로 기다릴 필요가 없을 때는 한 번의 메시지에서 여러 도구를 동시에 실행한다.

탐색 도구 선택 기준

상황	사용 도구
경로를 정확히 아는 경우	`Read` 직접 호출
특정 심볼 위치 확인	`Bash(grep -n)` 1회
넓은 코드베이스 탐색 (3회 이상 쿼리 예상)	`Agent(Explore)`
단순 디렉토리 구조 확인	`Bash(find -maxdepth 2)`

금지 패턴

- 같은 파일을 수정 전후로 반복 `Read`하기
- `find`로 탐색 → `grep`으로 좁히기 → `Read`로 확인 (3단계 이상 연쇄)
- 결과가 뻔한 명령어를 "확인용"으로 추가 실행하기
- 이미 읽은 파일을 다시 읽어 내용을 재확인하기

Agent 사용 기준

- 3회 이상의 쿼리가 필요한 탐색 → `Agent(Explore)` 위임
- 이미 답을 알고 있는 경우 → Agent 없이 직접 처리
- 단순 검색·단일 파일 수정 → Agent 사용 불필요
